Design and Analysis of Algorithms

Assignment 1 — Divide and Conquer & Asymptotic Notations

Name: _____ Group: _____

Prepared from the completed project and its measured benchmark data. Benchmark values below are actual outputs from the prepared project, not illustrative values.

1. Introduction

The project implements MergeSort, QuickSort and QuickSelect for integer arrays. It combines implementation, correctness testing, performance measurement and asymptotic analysis. The implementation uses a reusable MergeSort buffer, a 15-element cutoff, random-pivot 3-way QuickSort and one-sided QuickSelect.

2. Objectives

- Implement the three required algorithms.
- Measure time, comparisons and maximum recursion depth.
- Test random, sorted and duplicate-heavy inputs.
- Use JUnit 5 and edge-case tests.
- Export results to CSV and create time, depth and ratio plots.
- Use feature branches, meaningful commits and a v1.0 release.

3. Complexity Summary

Algorithm	Best	Average	Worst
MergeSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
QuickSort	$\Theta(n \log n)$ expected	$\Theta(n \log n)$ expected	$O(n^2)$
QuickSelect	$\Theta(n)$ expected	$\Theta(n)$ expected	$O(n^2)$
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$

MergeSort uses two recursive halves and linear merging. QuickSort depends on pivot balance; random pivots give $O(n \log n)$ expected behaviour, while very unbalanced partitions can give $O(n^2)$. QuickSelect continues on one side, so the balanced recurrence is $T(n)=T(n/2)+\Theta(n)=\Theta(n)$.

4. Recurrences

MergeSort: $T(n)=2T(n/2)+\Theta(n) \rightarrow \Theta(n \log n)$. QuickSort balanced case: $T(n)=2T(n/2)+\Theta(n) \rightarrow \Theta(n \log n)$. QuickSelect balanced case: $T(n)=T(n/2)+\Theta(n) \rightarrow \Theta(n)$.

5. Benchmark Method

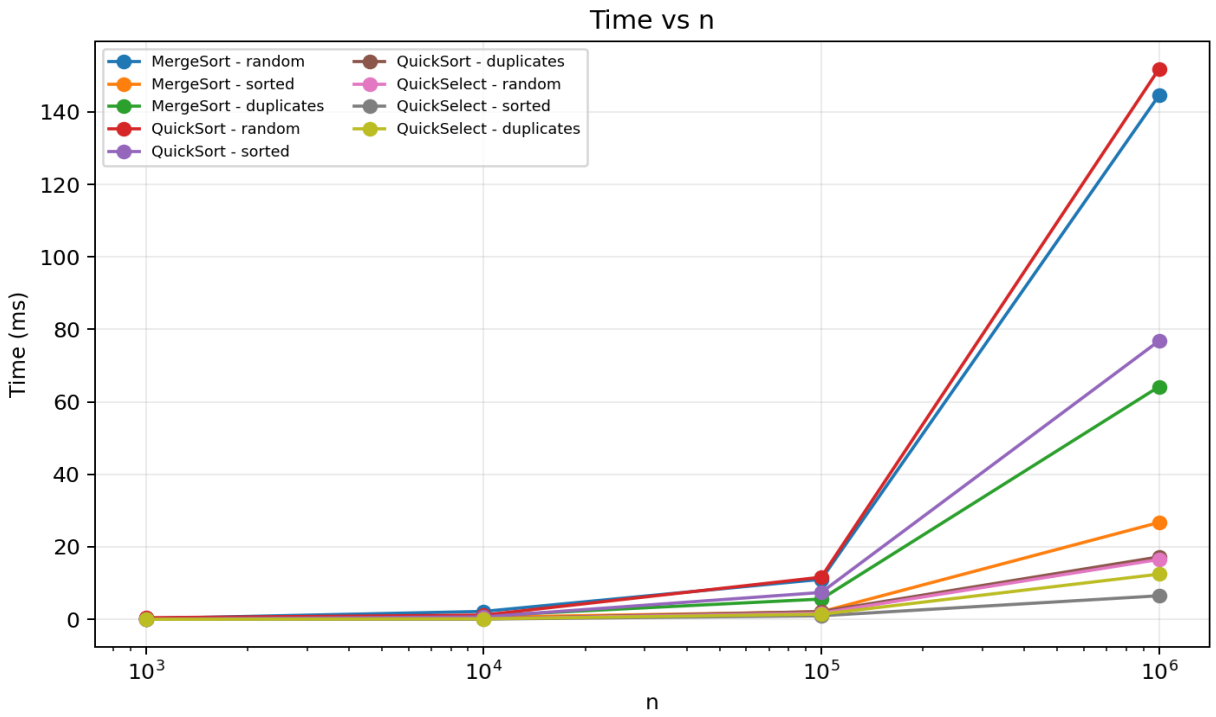
The benchmark uses n=1,000, 10,000, 100,000 and 1,000,000, with random, sorted and duplicate-heavy inputs. Each case is repeated five times and the median time is stored. results.csv contains algorithm,input,n,time_ms,comparisons,max_depth.

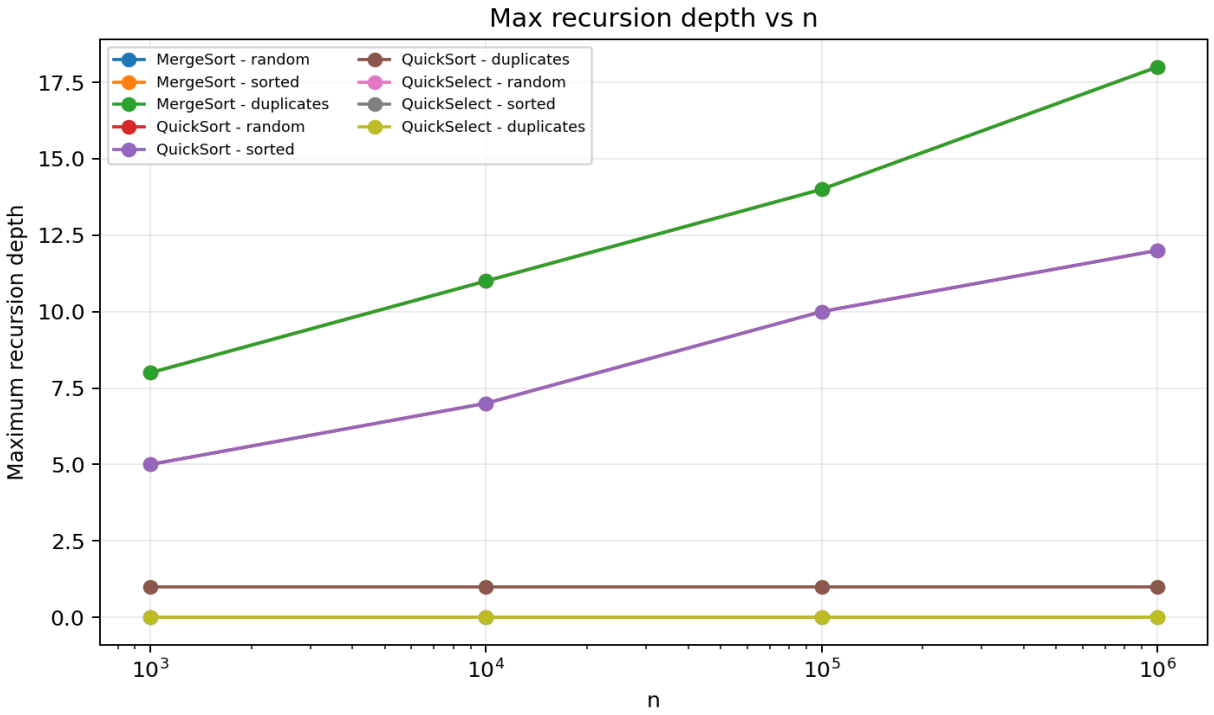
6. Measured Results

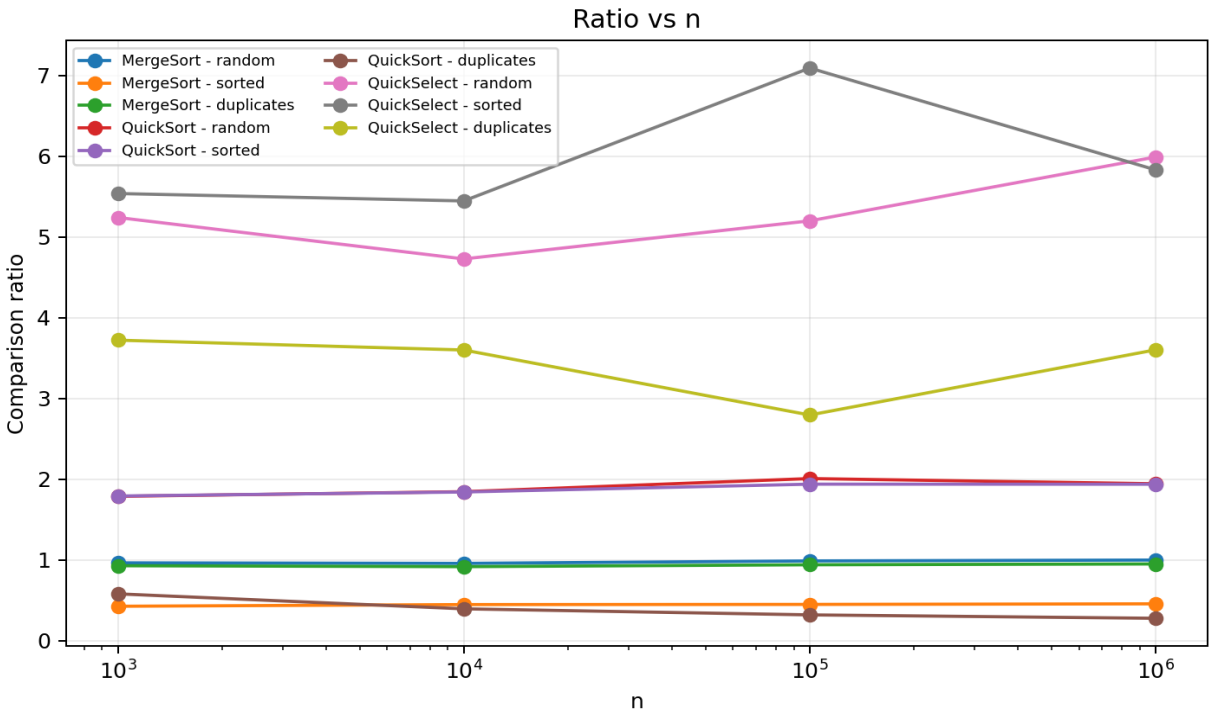
Algorithm	Input	n	Time ms	Depth
MergeSort	random	1,000,000	144.527	18
MergeSort	sorted	1,000,000	26.722	18
QuickSort	random	1,000,000	151.813	12
QuickSort	sorted	1,000,000	76.798	12
QuickSort	duplicates	1,000,000	17.218	1
QuickSelect	random	1,000,000	16.511	0
QuickSelect	sorted	1,000,000	6.496	0
QuickSelect	duplicates	1,000,000	12.459	0

At n=1,000,000, MergeSort/random took 144.527 ms, QuickSort/random took 151.813 ms and QuickSelect/random took 16.511 ms. QuickSort/duplicates had maximum depth 1, showing the effect of 3-way partitioning on equal values.

7. Plots




Max recursion depth vs n
MergeSort - random
MergeSort - sorted
MergeSort - duplicates
QuickSort - random
QuickSort - sorted
QuickSort - duplicates
QuickSelect - random
QuickSelect - sorted
QuickSelect - duplicates
Maximum recursion depth
n


Ratio vs n
MergeSort - random
MergeSort - sorted
MergeSort - duplicates
QuickSort - random
QuickSort - sorted
QuickSort - duplicates
QuickSelect - random
QuickSelect - sorted
QuickSelect - duplicates
Comparison ratio
n

8. Θ-Ratio Check

MergeSort / random: 0.963, 0.956, 0.987, 0.998

MergeSort / sorted: 0.425, 0.446, 0.448, 0.455

MergeSort / duplicates: 0.927, 0.916, 0.939, 0.949

QuickSort / random: 1.787, 1.845, 2.007, 1.943

QuickSort / sorted: 1.791, 1.841, 1.938, 1.937

QuickSort / duplicates: 0.579, 0.393, 0.318, 0.276

QuickSelect / random: 5.242, 4.729, 5.200, 5.992

QuickSelect / sorted: 5.539, 5.447, 7.094, 5.832

QuickSelect / duplicates: 3.722, 3.600, 2.795, 3.602

For sorting algorithms the ratio is comparisons/(n log■ n); for QuickSelect it is comparisons/n. The ratios remain bounded for the expected cases, which supports the theoretical growth rates. QuickSort on duplicate-heavy input has a decreasing ratio because the 3-way partition processes equal values together.

9. Testing

The project includes JUnit 5 tests for 100 random arrays, edge cases, QuickSelect correctness and the QuickSort depth condition. A separate local correctness runner was executed on 150 random cases plus edge checks and passed. The official Maven test command is mvn clean test.

10. Project Structure

Algorithm classes are separated from Metrics, Benchmark and tests. results.csv stores the measurements, plots/ stores PNG figures, README.md documents commands, and REPORT.pdf/docx provide the analysis. This separation keeps each responsibility clear and makes the code easier to test and defend.

11. Git Workflow

Required branches are feature/metrics, feature/mergesort, feature/quicksort and feature/select. Each contains a meaningful feature commit and is merged into main. The final repository is ready for the v1.0 tag after the report and benchmark files are committed. .gitignore excludes target and common IDE files.

12. Discussion and Conclusion

The measurements broadly follow the expected theoretical behaviour. MergeSort has logarithmic depth and stable n log n comparison growth. QuickSort keeps bounded recursion and benefits from 3-way partitioning on duplicates. QuickSelect uses fewer comparisons because it only follows the side containing k. Timing differences are affected by JVM warm-up, JIT compilation, garbage collection, CPU cache and other system activity, so the median of five runs is used.